Quick Security Evaluation

PeerDB Search

V 1.1
Amsterdam, February 8th, 2023
Confidential

1 Introduction

As part of the NLnet projects your project PeerDB Search has received a basic quick security evaluation from Radically Open Security. The goal of the review is to provide advice and input to consider in the further development of your project. The selected project gets 4 person days from ROS for a quick security evaluation.

2 Security Evaluation For PeerDB Search

Tasks Performed

- A review and static analysis of the source code and documentation at the public GitLab repository, https://gitlab.com/peerdb/search
- Discussed the project in a video conference with the developer
- Dynamic analysis of a public service instance, https://omni.peerdb.org/

Attack Surface and Threat Model

The application is a search engine that performs queries on structured data in an ElasticSearch database.

The data is imported from an administrator directly into the ElasticSearch database, parsing data that is provided to them, optionally using a custom script, from external sources. This data may be malicious, and it is not possible for an administrator to manually review all of it. The data stored in the database is not sensitive and does not contain any user secret, as all the data stored is meant to be accessible through the query engine to anyone. This reduces the impact of possible attacks meant to read data from the database, because there is no secret stored in it.

The users query the web front end, where the data stored in the ElasticSearch database is used to populate the web page, providing the end user with the queries results. The search engine is open and available to unregistered users and there is no user login or registration. User can only submit searches and cannot create content. This reduces the impact of client side attacks since there is no user content to compromise, and it is not possible for users to create content that may compromise the application.

The current attack surface is the following:

- Compromise of the PeerDB application with malicious imported search data. It might be possible that the data used to feed the ElasticSearch database contains malicious data that, when loaded in the database or rendered in the web page, generate malicious behavior in the application.
- Web-based attacks that can compromise the availability of the PeerDB web application. Since the search engine is open publicly on the internet, malicious users could try to connect and perform attacks that create heavy loads on the system, ultimately preventing legitimate user to access the system.

- Compromise of the ElasticSearch database. If the database configuration does not follow the security standards, it may be possible for an attacker to modify the stored data that could be later included in the web page.

3 Issues

Injection of Malicious HTML Content

The web application includes HTML content, in the page body. This content is originated from the data imported by the system administrator from external sources.

This happens when the application is using the `Vue.js` feature `v-html` that includes HTML content into the page without encoding. The feature usages are in the following locations:

- File `src/views/DocumentGet.vue` at line 162: `<div class="prose prose-slate max-w-none" v-html="article.html.en"></div>`
- File `src/components/PropertiesRows.vue` at line 60: `v-html="claim.html?.en"`
- File `src/components/SearchResult.vue` at line 118: `<p v-if="description" class="prose prose-slate max-w-none" v-html="description"></p>`

In this way, malicious HTML content could be included in the web page from the data that was used to populate the ElasticSearch database. This could lead to an attacker taking control of the web page through Cross-Site Scripting (XSS), displaying malicious content or executing JavaScript code.

It is always good practice to never include HTML code directly into the web page content. Since the application relies on HTML pages to mimic the original source page structure we suggest sanitizing all HTML content in the back-end before sending the data to the web client, additionally sanitizing all the content in the client. These sanitizing steps should be done via secure libraries, such as bluemonday for the server, and DOMPurify for the client.

Lack of Rate Limiting

The application handles request from anonymous users that could try to undermine the service availability. Since there is no rate limiting on the user requests an attacker could achieve a denial of service just by performing many heavy load queries for a long period of time. Two other possible way to achieve a denial of service were found and are reported below.

1. The in memory data structure caching user queries can grow without limitations and can be inflated by users, creating an opportunity for an attacker to exhaust the server memory.
   The application uses a `sync.Map` object named `searches` to cache in the server memory, the user searches. This object is accessed in the following files:

   - File document_search_amount_filter.go at line 88: `ss, ok := searches.Load(id)`
   - File document_search.go at line 430: `ps, ok := searches.Load(parentSearchID)`
   - File document_search.go at line 451: `searches.Store(sh.ID, sh)`
   - File document_search.go at line 464: `sh, ok := searches.Load(searchID)`
   - File document_search.go at line 491: `searches.Store(ss.ID, ss)`
   - File document_search.go at line 504: `sh, ok := searches.Load(searchID)`
   - File document_search_rel_filter.go at line 50: `ss, ok := searches.Load(id)`
   - File document_search_index_filter.go at line 41: `ss, ok := searches.Load(id)`
   - File document_search_time_filter.go at line 71: `ss, ok := searches.Load(id)`
   - File document_search_size_filter.go at line 46: `ss, ok := searches.Load(id)`
   - File document_search_filters.go at line 100: `ss, ok := searches.Load(id)`
   - File document_search_string_filter.go at line 46: `ss, ok := searches.Load(id)`

   In all this references the cache entries are not removed and since the cache itself does not have any timeout, the data will remain in the memory indefinitely until the server is restarted. An attacker could make a server crash at will by exhausting the system memory, but the time required to crash a server depends on the resources and configuration of the system.
   It is recommended to create a thread safe counter to enforce a maximum amount of entries in memory, and implement an expiration time for cache entries. In general, avoid to store unlimited data in the memory and use a profile to identify possible functions consuming memory without limitations.
2. The HTTP server that serves the web application, initialized at line 170 of file cmd/search/serve.go does not set the values for `http.Server` properties `ReadTimeout`, `ReadHeaderTimeout`, `WriteTimeout`, `IdleTimeout`. This means that the default ones are used, making those timeouts unbounded. A malicious user could exhaust the server resources by known HTTP attacks like slow read and slow write of the TCP connections. We suggest using a maximum value for those properties. These maximum values need to be evaluated depending on the average application processing times. Other solutions may include the use of HTTP proxies or load balancer to enforce rate limitations on client requests.

Lack of ElasticSearch Authentication

The ElasticSearch database does not use authentication, this could create a situation where the database is exposed and user can upload malicious data. The example configuration in the documentation uses docker to create a network where the application and the ElasticSearch database can connect, and no external user can connect to it. But if a user uses a different deployment or exposes the ElasticSearch database port to other networks than the docker one, it may

be possible to reach the ElasticSearch database TCP port. Furthermore, if the application is vulnerable to Server-Side Request Forgery (SSRF) it could be possible for an attacker to send requests to the ElasticSearch, inside the docker network, modifying its data.

We suggest changing the application code to authentication to the ElasticSearch service with credentials. It is also suggested to use two different users, one that has write privileges when importing data, only used by the admin, and a second user that only has read permissions used for the application to read the data.

4 Contact details

Your pentester(s) for this project:

- Andrea Jegher
  Andrea Jegher is an information security engineer focused on applications and infrastructure security. Andrea holds a Master Degree in Computer Science and Engineering from Politecnico di Milano, Italy.

If you have any questions about this advice, please contact us at info@radicallyopensecurity.com

For more information about Radically Open Security and its services please visit our website: www.radicallyopensecurity.com .

5 Disclaimer

This evaluation is not to be considered a full audit or pentest. It is important to understand the limits of ROS' services. ROS does not (and cannot) give guarantees that something is secure. Additionally, the above advice is obviously incomparable to a full-blown security audit as performed by ROS or any other professional security company, which takes orders of magnitude more time and effort. (Such an audit may be necessary in the future still, and we would be happy to work with you on that).

We recommend for now you treat our feedback as a discreet sanity check by knowledgeable friends; it is not the intention to publicly claim that Radically Open Security audited your code and found it to be safe or 'found no problems'. Rather, the intention is the reverse: we aim to help you capture obvious flaws within the very limited terms of this deal, and to protect the general audience from irresponsible projects putting them at risk. NGI Zero takes the public interest very seriously and wants to have at least a rough understanding of the maturity of your work - hopefully this will guide your own behavior in terms of any claims you make.

There is no shame in clearly messaging to users that your project is still in an early stage, that they use it at their own risk and that there are no guarantees - being honest with your users can only ever be a good thing.

Front page image by PickPik (https://www.pickpik.com/), "Dogs paw on the wooden keyboard", Image styling by Patricia Piolon, royalty-free stock photo.